

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 5**



SORTING

Oleh:

Muhammad Rafly Ash-Shadiq

NIM. 2510817310007

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 5

Laporan Praktikum Algoritma & Struktur Data Modul 5: Sorting ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Rafly Ash-Shadiq
NIM : 2510817310007

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Fadhil Syahdama Mahatma Putra
NIM. 2410817210026

Andreyan Rizky Baskara, S.Kom., M.Kom
NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR	iii
DAFTAR TABEL	iv
SOAL 1	1
A. Source Code	1
B. Output Program	9
C. Pembahasan	12
LINK GITHUB	17

DAFTAR GAMBAR

Gambar 1 Screenshot Output Program Insertion Sort.....	9
Gambar 2 Screenshot Output Program Merge Sort	9
Gambar 3 Screenshot Output Program Shell Sort.....	10
Gambar 4 Screenshot Output Program Quick Sort.....	10
Gambar 5 Screenshot Output Program Bubble Sort	11
Gambar 6 Screenshot Output Program Selection Sort.....	11
Gambar 7 Screenshot Output Program Keluar.....	11

DAFTAR TABEL

Table 1 Source Code Soal Nomor 1.....	1
---------------------------------------	---

SOAL 1

Lengkapi coding pada function tambahDepanH() agar bisa berjalan dengan lancar. running, simpan program, dan screenshoot hasil running !

A. Source Code

Table 1 Source Code Soal Nomor 1

1.	#include <iostream>
2.	#include <string>
3.	#include <conio.h>
4.	
5.	using namespace std;
6.	void insertionSort(string data[], int jumlah) {
7.	for(int i = 1; i < jumlah; i++) {
8.	string sementara = data[i];
9.	int j = i - 1;
10.	
11.	while(j >= 0 && data[j] > sementara) {
12.	data[j + 1] = data[j];
13.	j--;
14.	}
15.	
16.	data[j + 1] = sementara;
17.	}
18.	}
19.	
20.	void merge(string data[], int kiri, int tengah, int kanan) {
21.	
22.	int jumlahKiri = tengah - kiri + 1;
23.	int jumlahKanan = kanan - tengah;
24.	
25.	string arrKiri[100], arrKanan[100];
26.	
27.	for(int i = 0; i < jumlahKiri; i++) {
28.	arrKiri[i] = data[kiri + i];
29.	}
30.	

```

31.     for(int j = 0; j < jumlahKanan; j++) {
32.         arrKanan[j] = data[tengah + 1 + j];
33.     }
34.
35.     int i = 0;
36.     int j = 0;
37.     int k = kiri;
38.
39.     while(i < jumlahKiri && j < jumlahKanan) {
40.
41.         if(arrKiri[i] <= arrKanan[j]) {
42.             data[k] = arrKiri[i];
43.             i++;
44.         } else {
45.             data[k] = arrKanan[j];
46.             j++;
47.         }
48.
49.         k++;
50.     }
51.
52.     while(i < jumlahKiri) {
53.         data[k] = arrKiri[i];
54.         i++;
55.         k++;
56.     }
57.
58.     while(j < jumlahKanan) {
59.         data[k] = arrKanan[j];
60.         j++;
61.         k++;
62.     }
63. }
64.
65. void mergeSort(string data[], int kiri, int kanan) {
66.
67.     if(kiri < kanan) {
68.
69.         int tengah = (kiri + kanan) / 2;
70.

```

```

71.         mergeSort(data, kiri, tengah);
72.         mergeSort(data, tengah + 1, kanan);
73.
74.         merge(data, kiri, tengah, kanan);
75.     }
76. }
77.
78. void shellSort(string data[], int jumlah) {
79.
80.     for(int jarak = jumlah / 2; jarak > 0; jarak /=
2) {
81.
82.         for(int i = jarak; i < jumlah; i++) {
83.
84.             string simpan = data[i];
85.             int j;
86.
87.             for(j = i; j >= jarak && data[j - jarak]
> simpan; j -= jarak) {
88.                 data[j] = data[j - jarak];
89.             }
90.
91.             data[j] = simpan;
92.         }
93.     }
94. }
95.
96. int partition(int angka[], int awal, int akhir) {
97.
98.     int pivot = angka[akhir];
99.     int indeksKecil = awal - 1;
100.
101.     for(int i = awal; i < akhir; i++) {
102.
103.         if(angka[i] < pivot) {
104.
105.             indeksKecil++;
106.
107.             int tukar = angka[indeksKecil];
108.             angka[indeksKecil] = angka[i];

```



```

109.         angka[i] = tukar;
110.     }
111. }
112.
113.     int tukar = angka[indeksKecil + 1];
114.     angka[indeksKecil + 1] = angka[akhir];
115.     angka[akhir] = tukar;
116.
117.     return indeksKecil + 1;
118. }
119.
120. void quickSort(int angka[], int awal, int akhir) {
121.
122.     if(awal < akhir) {
123.
124.         int posisiPivot = partition(angka, awal,
125.         akhir);
126.
127.         quickSort(angka, awal, posisiPivot - 1);
128.         quickSort(angka, posisiPivot + 1, akhir);
129.     }
130. }
131. void bubbleSort(int angka[], int jumlah) {
132.
133.     for(int i = 0; i < jumlah - 1; i++) {
134.
135.         for(int j = 0; j < jumlah - i - 1; j++) {
136.
137.             if(angka[j] > angka[j + 1]) {
138.
139.                 int tukar = angka[j];
140.                 angka[j] = angka[j + 1];
141.                 angka[j + 1] = tukar;
142.             }
143.         }
144.     }
145. }
146.
147. void selectionSort(int angka[], int jumlah) {

```

```

148.
149.     for(int i = 0; i < jumlah - 1; i++) {
150.
151.         int posisiMin = i;
152.
153.         for(int j = i + 1; j < jumlah; j++) {
154.
155.             if(angka[j] < angka[posisiMin]) {
156.                 posisiMin = j;
157.             }
158.         }
159.
160.         int tukar = angka[i];
161.         angka[i] = angka[posisiMin];
162.         angka[posisiMin] = tukar;
163.     }
164. }
165.
166. void tampilNama(string data[], int jumlah) {
167.
168.     for(int i = 0; i < jumlah; i++) {
169.         cout << data[i] << " ";
170.     }
171.
172.     cout << endl;
173. }
174.
175. void tampilNIM(int data[], int jumlah) {
176.
177.     for(int i = 0; i < jumlah; i++) {
178.         cout << data[i] << " ";
179.     }
180.
181.     cout << endl;
182. }
183.
184. int main() {
185.
186.     int menu;
187.

```

```

188.     do {
189.
190.         system("cls");
191.
192.         cout                                     <<
193.         "===== "                               <<
194.         endl;
195.         cout << "          PROGRAM ALGORITMA SORTING
196.         " << endl;
197.         cout                                     <<
198.         "===== "                               <<
199.         endl;
200.         cout << "1. Insertion Sort (Nama)" << endl;
201.         cout << "2. Merge Sort      (Nama)" << endl;
202.         cout << "3. Shell Sort      (Nama)" << endl;
203.         cout << "4. Quick Sort      (NIM)" << endl;
204.         cout << "5. Bubble Sort     (NIM)" << endl;
205.         cout << "6. Selection Sort (NIM)" << endl;
206.         cout << "7. Keluar" << endl;
207.         cout                                     <<
208.         "===== "                               <<
209.         endl;
210.         cout << "Masukkan Pilihan : ";
211.         cin  >> menu;
212.
213.         if(menu >= 1 && menu <= 3) {
214.
215.             int jumlahNama;
216.
217.             cout << "\nJumlah Huruf>Nama : ";
218.             cin  >> jumlahNama;
219.
220.             string nama[100];
221.
222.             cout << "Masukkan Data Nama : " << endl;
223.
224.             for(int i = 0; i < jumlahNama; i++) {
225.                 cout << "Data ke-" << i + 1 << " : ";
226.                 cin  >> nama[i];
227.             }

```

```

221.
222.         cout << "\nData Sebelum Sorting : ";
223.         tampilNama(nama, jumlahNama);
224.
225.         if(menu == 1) {
226.
227.             insertionSort(nama, jumlahNama);
228.
229.             cout << "Hasil Insertion Sort : ";
230.             tampilNama(nama, jumlahNama);
231.
232.         } else if(menu == 2) {
233.
234.             mergeSort(nama, 0, jumlahNama - 1);
235.
236.             cout << "Hasil Merge Sort : ";
237.             tampilNama(nama, jumlahNama);
238.
239.         } else if(menu == 3) {
240.
241.             shellSort(nama, jumlahNama);
242.
243.             cout << "Hasil Shell Sort : ";
244.             tampilNama(nama, jumlahNama);
245.         }
246.     }
247.
248.     else if(menu >= 4 && menu <= 6) {
249.
250.         int jumlahDigit;
251.
252.         cout << "\nJumlah Angka NIM : ";
253.         cin >> jumlahDigit;
254.
255.         int nim[100];
256.
257.         cout << "Masukkan Digit NIM : " << endl;
258.
259.         for(int i = 0; i < jumlahDigit; i++) {

```

```

260.         cout << "Angka ke-" << i + 1 << " :
";
261.         cin  >> nim[i];
262.     }
263.
264.     cout << "\nData Sebelum Sorting : ";
265.     tampilNIM(nim, jumlahDigit);
266.
267.     if(menu == 4) {
268.
269.         quickSort(nim, 0, jumlahDigit - 1);
270.
271.         cout << "Hasil Quick Sort : ";
272.         tampilNIM(nim, jumlahDigit);
273.
274.     } else if(menu == 5) {
275.
276.         bubbleSort(nim, jumlahDigit);
277.
278.         cout << "Hasil Bubble Sort : ";
279.         tampilNIM(nim, jumlahDigit);
280.
281.     } else if(menu == 6) {
282.
283.         selectionSort(nim, jumlahDigit);
284.
285.         cout << "Hasil Selection Sort : ";
286.         tampilNIM(nim, jumlahDigit);
287.     }
288. }
289.
290.     cout  <<  "\nTekan  tombol  apapun  untuk
melanjutkan...";
291.     getch();
292.
293.     } while(menu != 7);
294.
295.     cout << "\nTerima kasih telah menggunakan program
ini." << endl;
296.

```

```

297.     return 0;
298. }

```

B. Output Program



```

=====
PROGRAM ALGORITMA SORTING
=====
1. Insertion Sort (Nama)
2. Merge Sort (Nama)
3. Shell Sort (Nama)
4. Quick Sort (NIM)
5. Bubble Sort (NIM)
6. Selection Sort (NIM)
7. Keluar
=====
Masukkan Pilihan : 1

Jumlah Huruf>Nama : 5
Masukkan Data Nama :
Data ke-1 : R
Data ke-2 : A
Data ke-3 : F
Data ke-4 : L
Data ke-5 : Y

Data Sebelum Sorting : R A F L Y
Hasil Insertion Sort : A F L R Y

Tekan tombol apapun untuk melanjutkan...

```

Gambar 1 Screenshot Output Program Insertion Sort



```

=====
PROGRAM ALGORITMA SORTING
=====
1. Insertion Sort (Nama)
2. Merge Sort (Nama)
3. Shell Sort (Nama)
4. Quick Sort (NIM)
5. Bubble Sort (NIM)
6. Selection Sort (NIM)
7. Keluar
=====
Masukkan Pilihan : 2

Jumlah Huruf>Nama : 7
Masukkan Data Nama :
Data ke-1 : A
Data ke-2 : S
Data ke-3 : A
Data ke-4 : Y
Data ke-5 : O
Data ke-6 : M
Data ke-7 : I

Data Sebelum Sorting : A S A Y O M I
Hasil Merge Sort : A A I M O S Y

Tekan tombol apapun untuk melanjutkan...

```

Gambar 2 Screenshot Output Program Merge Sort

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
=====
PROGRAM ALGORITMA SORTING
=====
1. Insertion Sort (Nama)
2. Merge Sort (Nama)
3. Shell Sort (Nama)
4. Quick Sort (NIM)
5. Bubble Sort (NIM)
6. Selection Sort (NIM)
7. Keluar
=====
Masukkan Pilihan : 3

Jumlah Huruf>Nama : 5
Masukkan Data Nama :
Data ke-1 : A
Data ke-2 : R
Data ke-3 : A
Data ke-4 : G
Data ke-5 : I

Data Sebelum Sorting : A R A G I
Hasil Shell Sort : A A G I R

Tekan tombol apapun untuk melanjutkan...
```

Gambar 3 Screenshot Output Program Shell Sort

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
=====
PROGRAM ALGORITMA SORTING
=====
1. Insertion Sort (Nama)
2. Merge Sort (Nama)
3. Shell Sort (Nama)
4. Quick Sort (NIM)
5. Bubble Sort (NIM)
6. Selection Sort (NIM)
7. Keluar
=====
Masukkan Pilihan : 4

Jumlah Angka NIM : 13
Masukkan Digit NIM :
Angka ke-1 : 2
Angka ke-2 : 5
Angka ke-3 : 1
Angka ke-4 : 0
Angka ke-5 : 8
Angka ke-6 : 1
Angka ke-7 : 7
Angka ke-8 : 3
Angka ke-9 : 1
Angka ke-10 : 0
Angka ke-11 : 0
Angka ke-12 : 0
Angka ke-13 : 7

Data Sebelum Sorting : 2 5 1 0 8 1 7 3 1 0 0 0 7
Hasil Quick Sort : 0 0 0 0 1 1 1 2 3 5 7 7 8

Tekan tombol apapun untuk melanjutkan...
```

Gambar 4 Screenshot Output Program Quick Sort

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
=====
PROGRAM ALGORITMA SORTING
=====
1. Insertion Sort (Nama)
2. Merge Sort (Nama)
3. Shell Sort (Nama)
4. Quick Sort (NIM)
5. Bubble Sort (NIM)
6. Selection Sort (NIM)
7. Keluar
=====
Masukkan Pilihan : 5

Jumlah Angka NIM : 7
Masukkan Digit NIM :
Angka ke-1 : 25
Angka ke-2 : 10
Angka ke-3 : 81
Angka ke-4 : 73
Angka ke-5 : 10
Angka ke-6 : 00
Angka ke-7 : 7

Data Sebelum Sorting : 25 10 81 73 10 0 7
Hasil Bubble Sort : 0 7 10 10 25 73 81

Tekan tombol apapun untuk melanjutkan...
```

Gambar 5 Screenshot Output Program Bubble Sort

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
=====
PROGRAM ALGORITMA SORTING
=====
1. Insertion Sort (Nama)
2. Merge Sort (Nama)
3. Shell Sort (Nama)
4. Quick Sort (NIM)
5. Bubble Sort (NIM)
6. Selection Sort (NIM)
7. Keluar
=====
Masukkan Pilihan : 6

Jumlah Angka NIM : 5
Masukkan Digit NIM :
Angka ke-1 : 251
Angka ke-2 : 0
Angka ke-3 : 81
Angka ke-4 : 73
Angka ke-5 : 10007

Data Sebelum Sorting : 251 0 81 73 10007
Hasil Selection Sort : 0 73 81 251 10007

Tekan tombol apapun untuk melanjutkan...
```

Gambar 6 Screenshot Output Program Selection Sort

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
=====
PROGRAM ALGORITMA SORTING
=====
1. Insertion Sort (Nama)
2. Merge Sort (Nama)
3. Shell Sort (Nama)
4. Quick Sort (NIM)
5. Bubble Sort (NIM)
6. Selection Sort (NIM)
7. Keluar
=====
Masukkan Pilihan : 7

Tekan tombol apapun untuk melanjutkan...
Terima kasih telah menggunakan program ini.
PS C:\Users\Lenovo 10Q\OneDrive\Documents\PRAKTIKUM ALGO_1\Modul_5_Algo>
```

Gambar 7 Screenshot Output Program Keluar

C. Pembahasan

Pada baris 1 `#include <iostream>` digunakan untuk menjalankan proses input dan output pada program seperti `cin` dan `cout`. Library ini sangat penting karena seluruh interaksi program dengan pengguna dilakukan melalui terminal. Pada baris 2 `#include <string>` digunakan agar program dapat menggunakan tipe data `string` untuk menyimpan data nama. Tanpa library ini, program tidak dapat memproses teks atau karakter dalam bentuk kata. Pada baris 3 `#include <conio.h>` digunakan untuk menjalankan fungsi `getch()` yang berfungsi menahan layar program agar tidak langsung tertutup setelah proses selesai dijalankan. Pada baris 5 `using namespace std;` digunakan agar penulisan `syntax` menjadi lebih sederhana tanpa perlu menuliskan `std::` di depan `cin`, `cout`, atau `string`.

Pada baris 6–18 dideklarasikan function `insertionSort()` yang digunakan untuk mengurutkan data nama menggunakan algoritma Insertion Sort secara ascending. Pada baris 7 dilakukan perulangan `for` dimulai dari indeks ke-1 karena elemen pertama dianggap sudah terurut. Pada baris 8 variabel sementara digunakan untuk menyimpan data yang sedang dibandingkan. Pada baris 9 variabel `j` digunakan sebagai indeks pembanding dengan elemen sebelumnya. Pada baris 11 dilakukan pengecekan apakah data sebelumnya lebih besar dari data sementara. Jika kondisi benar, maka pada baris 12 data digeser ke kanan agar memberi ruang untuk data yang lebih kecil. Pada baris 13 indeks `j` dikurangi agar proses pembandingan terus bergerak ke kiri. Setelah posisi yang tepat ditemukan, pada baris 16 data sementara dimasukkan ke posisi yang sesuai. Algoritma ini bekerja seperti proses menyusun kartu secara manual, di mana data akan disisipkan ke posisi yang tepat secara bertahap.

Pada baris 20–63 dibuat function `merge()` yang digunakan pada algoritma Merge Sort untuk menggabungkan dua bagian array yang sudah terurut. Pada baris 22 dan 23 dihitung jumlah elemen bagian kiri dan kanan array. Pada baris 25 dibuat array sementara `arrKiri` dan `arrKanan` untuk menyimpan data sementara selama proses penggabungan. Pada baris 27–33 data dari array utama disalin ke array kiri dan kanan.

Pada baris 35–37 dibuat variabel indeks i , j , dan k untuk mengontrol posisi array kiri, kanan, dan array utama. Pada baris 39 dilakukan perulangan selama kedua array masih memiliki data. Pada baris 41 dilakukan perbandingan antara data kiri dan kanan. Jika data kiri lebih kecil atau sama, maka data dimasukkan ke array utama pada baris 42 dan indeks kiri bertambah pada baris 43. Jika tidak, data kanan dimasukkan pada baris 45 dan indeks kanan bertambah pada baris 46. Pada baris 52–62 sisa data yang belum dipindahkan akan dimasukkan seluruhnya ke array utama. Function ini menunjukkan konsep utama Merge Sort yaitu membagi array menjadi bagian kecil lalu menggabungkannya kembali dalam keadaan terurut.

Pada baris 65–76 dibuat function `mergeSort()` yang merupakan inti recursive dari algoritma Merge Sort. Pada baris 67 dilakukan pengecekan apakah array masih bisa dibagi. Pada baris 69 ditentukan titik tengah array menggunakan $(\text{kiri} + \text{kanan}) / 2$. Pada baris 71 dan 72 function memanggil dirinya sendiri untuk membagi bagian kiri dan kanan array secara terus menerus hingga hanya tersisa satu elemen. Setelah proses pembagian selesai, pada baris 74 function `merge()` dipanggil untuk menggabungkan kembali data dalam keadaan terurut. Relasi antara `mergeSort()` dan `merge()` sangat penting karena `mergeSort()` bertugas membagi data sedangkan `merge()` bertugas menyusun kembali data tersebut.

Pada baris 78–94 dibuat function `shellSort()` yang digunakan untuk mengurutkan data menggunakan algoritma Shell Sort. Pada baris 80 ditentukan nilai jarak atau gap awal yaitu setengah dari jumlah data. Nilai gap ini akan terus diperkecil hingga menjadi 1. Pada baris 82 dilakukan perulangan untuk membandingkan data dengan jarak tertentu. Pada baris 84 variabel `simpan` digunakan untuk menyimpan data sementara. Pada baris 87 dilakukan pengecekan apakah data sebelumnya lebih besar dari data yang disimpan. Jika benar, maka data digeser pada baris 88. Setelah posisi yang sesuai ditemukan, data sementara dimasukkan kembali pada baris 91. Shell Sort merupakan pengembangan dari Insertion Sort, tetapi menggunakan jarak tertentu agar proses pengurutan menjadi lebih cepat pada data besar.

Pada baris 96–118 dibuat function `partition()` yang digunakan dalam algoritma Quick Sort. Pada baris 98 dipilih data terakhir sebagai pivot. Pivot merupakan titik acuan pembagian data. Pada baris 99 variabel `indeksKecil` digunakan untuk menandai posisi elemen yang lebih kecil dari pivot. Pada baris 101 dilakukan perulangan untuk membandingkan seluruh data dengan pivot. Jika data lebih kecil dari pivot pada baris 103, maka indeks kecil bertambah pada baris 105 dan dilakukan pertukaran data pada baris 107–109. Setelah seluruh data dibandingkan, pivot dipindahkan ke posisi yang tepat pada baris 113–115. Function ini menghasilkan posisi pivot yang sudah benar sehingga data kiri lebih kecil dan data kanan lebih besar dari pivot.

Pada baris 120–129 dibuat function `quickSort()` yang menjalankan algoritma Quick Sort secara recursive. Pada baris 122 dilakukan pengecekan apakah array masih dapat dibagi. Pada baris 124 dipanggil function `partition()` untuk menentukan posisi pivot. Setelah posisi pivot diperoleh, function memanggil dirinya sendiri pada bagian kiri dan kanan pivot melalui baris 126 dan 127. Dengan proses ini, array akan terus dibagi hingga seluruh data terurut. Relasi antara `quickSort()` dan `partition()` sangat erat karena `quickSort()` mengatur proses recursive sedangkan `partition()` menentukan pembagian data.

Pada baris 131–145 dibuat function `bubbleSort()` yang digunakan untuk mengurutkan data menggunakan Bubble Sort. Pada baris 133 dilakukan perulangan utama sebanyak jumlah data dikurangi satu. Pada baris 135 dilakukan perulangan pembandingan antar elemen yang berdekatan. Pada baris 137 dilakukan pengecekan apakah data kiri lebih besar dari data kanan. Jika benar, maka dilakukan pertukaran data pada baris 139–141. Algoritma ini bekerja dengan cara “menggelembungkan” data terbesar ke bagian akhir array pada setiap perulangan.

Pada baris 147–164 dibuat function `selectionSort()` yang digunakan untuk mengurutkan data dengan mencari nilai terkecil pada setiap iterasi. Pada baris 149 dilakukan perulangan utama untuk menentukan posisi data yang akan diisi. Pada baris

151 variabel posisiMin digunakan untuk menyimpan indeks data terkecil sementara. Pada baris 153–157 dilakukan pencarian data terkecil pada sisa array. Setelah data terkecil ditemukan, dilakukan pertukaran data pada baris 160–162. Algoritma ini bekerja dengan cara memilih data terkecil lalu menempatkannya di posisi paling depan secara bertahap.

Pada baris 166–173 dibuat function tampilNama() yang digunakan untuk menampilkan data string dalam array. Pada baris 168 dilakukan perulangan untuk menampilkan seluruh data satu per satu menggunakan cout. Pada baris 172 digunakan endl untuk pindah baris setelah seluruh data selesai ditampilkan. Function ini digunakan setelah proses input maupun setelah sorting selesai dilakukan.

Pada baris 175–182 dibuat function tampilNIM() yang memiliki fungsi hampir sama dengan tampilNama(), tetapi digunakan untuk menampilkan data integer berupa digit NIM. Perulangan dilakukan untuk menampilkan seluruh isi array angka ke layar.

Pada baris 184–297 dibuat function utama main() sebagai pusat pengendali seluruh program. Pada baris 188 digunakan perulangan do-while agar menu terus muncul sampai pengguna memilih keluar. Pada baris 192–203 program menampilkan menu pilihan algoritma sorting. Pada baris 204 input pilihan pengguna dimasukkan ke variabel menu.

Pada baris 206 dilakukan pengecekan apakah pengguna memilih algoritma sorting nama yaitu Insertion Sort, Merge Sort, atau Shell Sort. Pada baris 208 dibuat variabel jumlahNama untuk menentukan banyak data yang akan dimasukkan. Pada baris 213 dibuat array nama[100] untuk menyimpan data nama. Pada baris 217–220 dilakukan input data nama sesuai jumlah yang dimasukkan sebelumnya. Setelah data dimasukkan, pada baris 222–223 program menampilkan data sebelum sorting dilakukan. Selanjutnya program menjalankan algoritma sesuai pilihan menu dan menampilkan hasil pengurutan.

Pada baris 248 dilakukan pengecekan untuk algoritma sorting NIM yaitu Quick Sort, Bubble Sort, dan Selection Sort. Pada baris 250 dibuat variabel jumlahDigit untuk menentukan jumlah angka yang dimasukkan. Pada baris 255 dibuat array nim[100] untuk menyimpan data angka. Pada baris 259–262 dilakukan input data angka sesuai jumlah yang ditentukan. Setelah itu data ditampilkan sebelum sorting lalu diproses menggunakan algoritma yang dipilih pengguna.

Pada baris 290 digunakan getch() agar layar tidak langsung tertutup sehingga pengguna dapat melihat hasil sorting terlebih dahulu. Pada baris 293 perulangan akan terus berjalan sampai pengguna memilih menu 7 untuk keluar. Pada baris 295 program menampilkan pesan penutup sebelum program selesai dijalankan.

LINK GITHUB

<https://github.com/Arthur-369/PRAKTIKUM-Algoritma-dan-Struktur-Data>